<u>פרויקט ג' – בדיקת ביצועים לשאילתות עסקיות</u>

שמות חברי הצוות:

נאות דוכן

הודיה ארביב

גלי אהרון

<u>שלב 1 – ניסוח שאילתות עסקיות:</u>
לצורך תחקור ביצועי התשאול נבחרה שאילתה מספר 1 מתוך חלק ב' של הפרויקט. השאילתה כוללת חיבור בין טבלת ההזמנות לבין טבלת המסעדות, ולאחר החיבור מבוצע קיבוץ של הרשומות לפי העיר שבה נמצאת המסעדה.

query1 – שאילתה ראשונה

תיאור עסקי:

הצגת חמש הערים בעלות סך המכירות הגבוה ביותר. השאילתה מאפשרת לזהות באילו ערים נוצר היקף המכירות הכולל הגבוה ביותר, ובנוסף מציגה את מספר ההזמנות שבוצעו בכל עיר.

טבלאות המקור:

zomato_orders – טבלת ההזמנות.

zomato_restaurant – טבלת המסעדות.

עמודות המשמשות לחיבור:

zomato_orders.r_id

zomato_restaurant.id

החיבור מבוצע כאשר מזהה המסעדה בטבלת ההזמנות שווה למזהה המסעדה בטבלת המסעדות:

zomato_orders.r_id = zomato_restaurant.id

כדי להתאים את סוג הנתונים לסוג הנתונים r_id_long, ונשמרת בשם long מומרת בקוד לטיפוס r_id עמודת id של העמודה.

העמודה המשמשת לקיבוץ:

city – העיר שבה נמצאת המסעדה.

העמודות והפונקציות המשמשות לאגרגציה:

sales_amount – חישוב סך המכירות באמצעות SUM.

חישוב מספר הרשומות בכל קבוצה – COUNT.

התוצאות ממוינות לפי total_sales בסדר יורד ומוגבלות לחמש הערים בעלות סך המכירות הגבוה ביותר.

query2 – שאילתה שנייה

תיאור עסקי:

הצגת חמש הערים בעלות סכום ההזמנה הממוצע הגבוה ביותר. השאילתה מאפשרת לזהות באילו ערים הלקוחות מוציאים בממוצע סכום גבוה יותר בכל הזמנה.

טבלאות המקור:

zomato_orders – טבלת ההזמנות.

zomato_restaurant – טבלת המסעדות.

עמודות המשמשות לחיבור:

zomato_orders.r_id

zomato_restaurant.id

החיבור מבוצע כאשר:

zomato_orders.r_id = zomato_restaurant.id

העמודה המשמשת לקיבוץ:

city – העיר שבה נמצאת המסעדה.

העמודה והפונקציה המשמשות לאגרגציה:

sales_amount – חישוב AVG סכום ההזמנה הממוצע בכל עיר באמצעות.

התוצאות ממוינות לפי ממוצע סכום ההזמנה בסדר יורד ומוגבלות לחמש הערים בעלות הממוצע הגבוה ביותר.

הקשר בין שתי השאילתות

שתי השאילתות משתמשות באותן טבלאות, באותן עמודות חיבור ובאותה עמודת קיבוץ. ההבדל העיקרי ביניהן הוא פונקציית האגרגציה המופעלת על העמודה sales_amount.

query2 מחושב סכום ההזמנה הממוצע בכל עיר באמצעות AVG.

<u>שלב 2 – חיבור טבלאות:</u>

חיבור הטבלאות

לצורך ביצוע השאילתות נבחרו שתי טבלאות הבסיס:

zomato_orders

zomato_restaurant

החיבור בוצע באמצעות העמודות:

zomato_orders.r_id_long = zomato_restaurant.id

r_id לטיפוס long כדי שתתאים לטיפוס הנתונים של העמודה id. לפני ביצוע החיבור הומרה העמודה

גודל הטבלאות:

zomato_orders - 2.03 MB

zomato_restaurant - 5.00 MB

חמש השורות הראשונות של הטבלה המחוברת:



```
First five rows of the joined table:
+----------+------------+--------+-------+-------+---------------+---------+-----+-----------------------------+-----------------+------+------------+-----+-------------------------+
|order_date|sales_amount|currency|user_id|r_id   |avg_item_price|r_id_long|id   |name                         |city             |rating|rating_count|cost |cuisine                  |
+----------+------------+--------+-------+-------+---------------+---------+-----+-----------------------------+-----------------+------+------------+-----+-------------------------+
|2017-11-14|259         |INR\r   |77851  |2040.0 |259.0          |2040     |2040 |Goli Vada Pav No .1          |Jayanagar,Bangalore|4.1   |50+ ratings |₹ 100|Maharashtrian,North Indian|
|2018-12-06|2218        |INR\r   |30683  |2250.0 |739.33         |2250     |2250 |Karolbaug                    |Baner,Pune       |4.1   |100+ ratings|₹ 750|Mughlai,North Indian     |
|2020-01-17|83          |INR\r   |55866  |2509.0 |83.0           |2509     |2509 |Poetry By Love & Cheesecake  |Powai,Mumbai     |4.4   |500+ ratings|₹ 800|Bakery,Desserts          |
|2017-12-05|1690        |INR\r   |69166  |11619.0|153.64         |11619    |11619|The Rose Restaurant Lake Town|Lake Town,Kolkata|4.3   |100+ ratings|₹ 350|North Indian,Chinese     |
|2019-05-23|74          |INR\r   |87855  |13452.0|74.0           |13452    |13452|Yummy Parathas               |Sinhgad Road,Pune|4.5   |100+ ratings|₹ 250|Punjabi,North Indian     |
```

אסטרטגיות החיבור

נבדקו שלוש אסטרטגיות חיבור:

זמן ריצה 13.18 שניות – Sort Merge Join

זמן ריצה 1.35 שניות – Broadcast Hash Join

זמן ריצה 6.04 שניות – Shuffle Hash Join

אימות התוכנית הפיזית

התוכנית הפיזית של כל אחת מהשאילתות נבדקה באמצעות explain().

נמצא כי:

עבור Sort Merge Join הופיע האלגוריתם SortMergeJoin.

עבור Broadcast Join הופיע האלגוריתם BroadcastHashJoin.

עבור Shuffle Hash Join הופיע האלגוריתם ShuffledHashJoin.

השוואת ביצועים

מהבדיקה עולה כי:

היה החיבור המהיר ביותר עם זמן ריצה של 1.35 שניות Broadcast Hash Join.

היה איטי יותר וארך 13.18 שניות Sort Merge Join.

היה האיטי ביותר וארך 6.04 שניות Shuffle Hash Join.

נימוק לתוצאות

קטנה (2.03MBגודל zomato_orders) היה המהיר ביותר משום שאחת הטבלאות Broadcast Hash Join (broadcast) לכל ה-Executors, ולכן ניתן היה לשדר אותה zomato_restaurant (5.00MB), יחסית לעומת zomato_orders בהתאם לגודלה היחסי, בין המחיצות. בקוד שודרה טבלת Shuffle וכך נמנע שלב יקר של כנדרש.

דרש מיון של שתי הטבלאות לפני החיבור, ולכן זמן הריצה שלו היה ארוך יותר Sort Merge Join.

לאחר Hash Table של הנתונים בין המחיצות ובניית (Shuffle) ביצע חלוקה מחדש Shuffle Hash Join החלוקה, ולכן היה האיטי ביותר.

<u>שלב 3 – כתיבת הטבלה לתשאול:</u>

יצירת הטבלה לתשאול

החיבור בוצע בין zomato_restaurant ו־zomato_orders לצורך יצירת הטבלה לתשאול חוברו טבלאות הבסיס בטבלת המסעדות id בטבלת ההזמנות לבין העמודה r_id_long באמצעות התאמה בין העמודה.

בהתבסס על תוצאות בדיקת אסטרטגיות החיבור בשלב הקודם, נבחרה אסטרטגיית Broadcast Hash Join, אשר השיגה את זמן הריצה המהיר ביותר, 1.35 שניות. במסגרת החיבור שודרה טבלת zomato_orders broadcast. באמצעות רמז (הטבלה הקטנה מבין השתיים

הטבלה המחוברת נכתבה בשתי צורות:

data/table.parquet טבלה רגילה לנתיב.

data/partitioned_table.parquet לנתיב ,city טבלה המחולקת למחיצות לפי העמודה.

בחירת עמודת החלוקה

נבחרה כעמודת החלוקה מכיוון שהיא משמשת כעמודת הקיבוץ בשתי השאילתות העסקיות city העמודה שנבחרו. בשאילתה הראשונה מחושב סך המכירות לפי עיר, ובשאילתה השנייה מחושב סכום ההזמנה הממוצע לפי עיר.

מרכזת רשומות השייכות לאותה עיר באותה מחיצה לוגית בדיסק. מטרת החלוקה city חלוקת הטבלה לפי היא לבדוק בהמשך האם אחסון הנתונים בהתאם לעמודת הקיבוץ משפיע על זמן ביצוע השאילתות לעומת טבלה שאינה מחולקת למחיצות.

אופן הכתיבה

overwrite המאפשר אחסון עמודתי וקריאה יעילה של הנתונים. האפשרות ,Parquet הכתיבה בוצעה בפורמט נבחרה כדי לאפשר הרצה חוזרת של הסקריפט בלי לקבל שגיאה כאשר נתיבי הפלט כבר קיימים.

<u>שלב 4 – אחסון בדיסק:</u>

בדיקת אופן אחסון הטבלה בדיסק

בשלב זה נבדקה השפעת אופן אחסון הטבלה לתשאול על זמן ביצוע שתי השאילתות העסקיות. זמן הריצה שנמדד כלל את קריאת הטבלה מהדיסק ואת ביצוע שתי השאילתות יחד.

הסקריפט הורץ בשתי תצורות:

זמן ריצה 20.06 שניות – table.parquet

זמן ריצה 104.65 שניות – partitioned_table.parquet

אשר נקבע כזמן הבסיס להמשך התחקור, הוא 20.06 שניות ,table.parquet זמן הריצה של.

הטבלה הרגילה הייתה מהירה יותר מהטבלה המחולקת למחיצות. זמן הריצה של הטבלה המחולקת היה ארוך ב־84.59 שניות מזמן הבסיס.

נימוק להבדלים

עם זאת, שתי השאילתות מבצעות חולקה למחיצות לפי העמודה city. הטבלה partitioned_table.parquet עדיין נדרש Spark לכן .where או filter לפי כל ערכי העיר ואינן מסננות עיר מסוימת באמצעות groupBy לקרוא את כלל המחיצות ולא נהנה מ־Partition Pruning.

קטנים. קריאה של קבצים רבים כרוכה Parquet יצרה מספר רב של תיקיות וקובצי city בנוסף, חלוקה לפי וניהול משימות רבות. במקרה זה התקורה הייתה גבוהה metadata בתקורה של פתיחת קבצים, סריקת .מהיתרון האפשרי של חלוקת הנתונים, ולכן זמן הביצוע של הטבלה המחולקת היה ארוך יותר

מכיוון שהיא data/table.parquet עודכן כך שיקרא את הטבלה מתוך queries.py בהתאם לתוצאות, הסקריפט .סיפקה את זמן הביצוע המועדף

חישוב ההבדל

ההפרש הוא: 104.65 - 20.06 = 84.59 שניות

הטבלה המחולקת לקחה בערך פי: 104.65 / 20.06 ≈ 5.22

כלומר היא הייתה איטית בערך פי 5.22 מהטבלה הרגילה.

<u>שלב 5 – אחסון ביניים:</u>

בדיקת רמות אחסון ביניים

הבדיקות בוצעו .persist בשלב זה נבדקה השפעת שמירת מסגרת הנתונים באחסון ביניים באמצעות הפקודה .שנמצאה בשלב הקודם כמהירה יותר מהטבלה המחולקת למחיצות ,table.parquet על הטבלה

נבדקו שתי רמות אחסון:

.שמירת הנתונים בזיכרון בלבד – MEMORY_ONLY

.שמירת הנתונים בזיכרון, ובמקרה שאין די מקום, שימוש גם בדיסק – MEMORY_AND_DISK

שנוצר cacheכדי לצמצם את השפעת סדר ההרצות וה־ Spark, כל רמת אחסון נבדקה בהרצה נפרדת של .בהרצה קודמת

זמן ריצה 20.06 שניות – persist ללא

זמן ריצה 17.70 שניות – MEMORY_ONLY

זמן ריצה 38.89 שניות – MEMORY_AND_DISK

הייתה המהירה ביותר (17.70 שניות), ואחריה ההרצה MEMORY_ONLY מהתוצאות עולה כי ההרצה עם הייתה האיטית ביותר (38.89 שניות), איטית MEMORY_AND_DISK .ללא שמירת ביניים (20.06 שניות) .MEMORY_ONLY-וב־21.19 שניות מ persist ב־18.83 שניות מההרצה ללא

נימוק לתוצאות

,(הפרש של כ-2.4 שניות בלבד) persist היה מעט מהיר יותר מההרצה ללא MEMORY_ONLY ,בהרצה זו .היה איטי משמעותית משניהם MEMORY_AND_DISK בעוד

עשוי (שניות 17.70) MEMORY_ONLY לבין (שניות 38.89) MEMORY_AND_DISK ההבדל הגדול בין שלא ,MEMORY_AND_DISK-לנבוע מהתקורה של כתיבה לדיסק וניהול בלוקים בין הזיכרון לדיסק ב .MEMORY_ONLY-נדרשה ב

עודכן כך שמסגרת הנתונים נשמרת באמצעות queries.py בהתאם לתוצאות, הסקריפט משום שזו הייתה שיטת ההרצה המהירה ביותר מבין שלוש ,persist(StorageLevel.MEMORY_ONLY) .האפשרויות שנבדקו

שלב 6 – הקצאת משאבי זיכרון:

בדיקת השפעת הקצאת הזיכרון ל־JVM

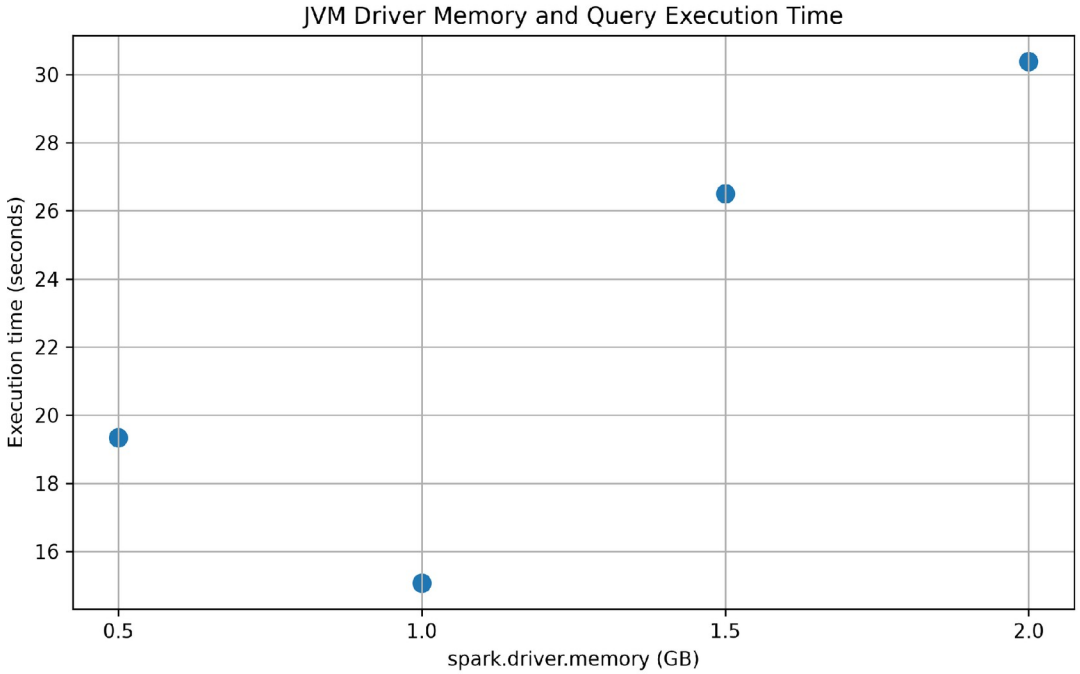
על זמן הביצוע הכולל של שתי השאילתות spark.driver.memory בשלב זה נבדקה השפעת הפרמטר GB: 500MB, 1GB, 1.5GB ו־2GB. נבדקו ארבע הקצאות זיכרון במרווחים שווים של 0.5 העסקיות.

זמן ריצה 19.34 שניות – 500m

זמן ריצה 15.07 שניות – 1g

זמן ריצה 26.50 שניות – 1500m

זמן ריצה 30.38 שניות – 2g



לבין זמן ביצוע שתי השאילתות JVM שהוקצה ל־Driver תרשים 1: הקשר בין זיכרון ה־.

עם זמן ריצה של 15.07 שניות. בהתאם, g הקצאת הזיכרון שהניבה את זמן הביצוע המהיר ביותר הייתה 1 g. מוגדר כ־1 spark.driver.memory עודכן כך שהפרמטר queries.py לכך, הסקריפט

ניתוח התוצאות

MB (19.34 הניב את זמן הביצוע הטוב ביותר (15.07 שניות), שיפור לעומת 500 GB מהתרשים עולה כי 1 GB זמן הריצה עלה ל-26.50 עם זאת, הקצאה גבוהה יותר לא המשיכה לשפר את הביצועים: ב-1.5 .(שניות GB שניות, וב-2 התקבל זמן ריצה גבוה במיוחד של 30.38 שניות.

נימוק לתוצאות

הקצאת זיכרון גדולה יותר אינה מבטיחה בהכרח שיפור בביצועים, במיוחד כאשר הטבלה והשאילתות אינן דורשות נפח זיכרון גדול. כאשר מוקצה זיכרון מעבר לצורך, עלולה להיווצר תקורה נוספת הקשורה לאתחול ניהול הזיכרון ואיסוף אשפה, JVM ה־.

זיכרון של Spark Driver-בנוסף, העבודה בוצעה בתוך מכונה עם משאבי זיכרון מוגבלים. כאשר הוגדר ל ייתכן שהמערכת המארחת נותרה עם פחות זיכרון פנוי, דבר שגרם להחלפת נתונים מול ,GBאו 1.5GB 2 הדיסק או לעומס על מערכת ההפעלה. לכן התקבלו זמני ביצוע ארוכים יותר עבור הקצאות אלו.

סיפקה במקרה זה איזון בין כמות זיכרון מספקת לביצוע השאילתות לבין הימנעות מהקצאת GBהקצאה של 1 יתר.

<u>שלב 7 – הקצאת רכיבי זיכרון:</u>

בדיקת הקצאת רכיבי זיכרון

ושל הפרמטר spark.memory.fraction בשלב זה נבדקה ההשפעה המשולבת של הפרמטר spark.memory.storageFraction על זמן ביצוע שתי השאילתות העסקיות.

spark.memory.fraction: נבדקו שלושה ערכים עבור 0.6, 0.7 ו־0.8. עבור כל אחד מהם נבדקו שלושה ערכים של spark.memory.storageFraction: 0.4, 0.5 ו־0.6. בסך הכול בוצעו תשע הרצות.

חושב לפי הנוסחה executionFraction הערך:

executionFraction = 1 - storageFraction

הצירוף שהניב את זמן הביצוע המהיר ביותר היה:

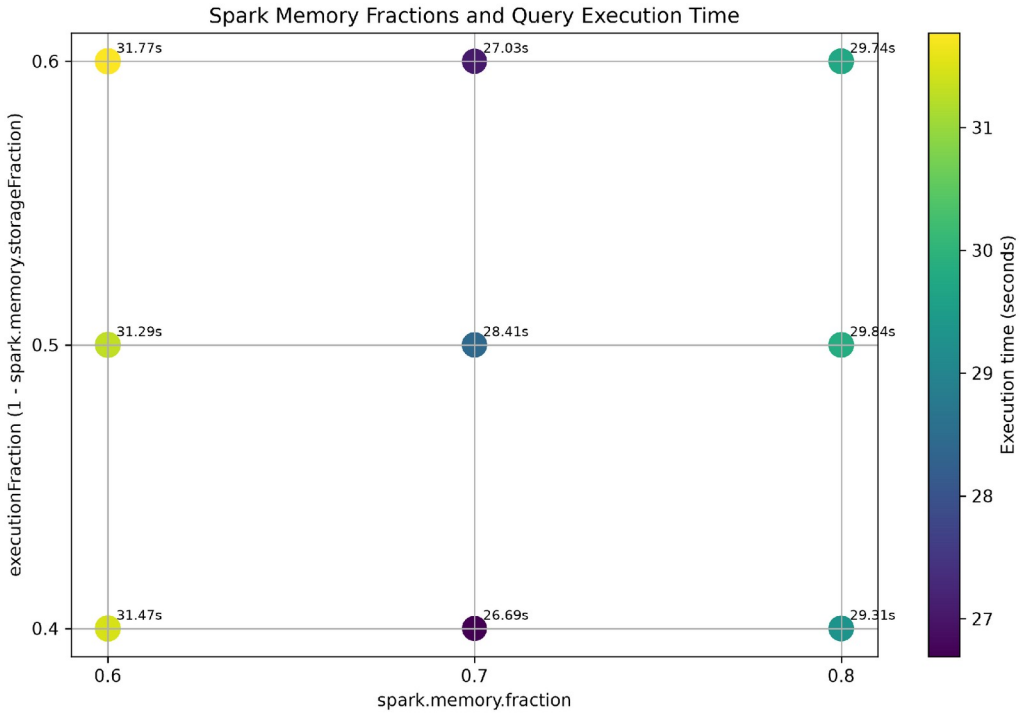
spark.memory.fraction = 0.7

spark.memory.storageFraction = 0.6

executionFraction = 0.4

זמן ריצה = 26.69 שניות

spark.memory.fraction מוגדר ל־0.7 עודכן כך שהפרמטר queries.py בהתאם לתוצאות, הסקריפט spark.memory.storageFraction מוגדר ל־0.6 והפרמטר.



תרשים 2: הקשר בין spark.memory.fraction ו-spark.memory.storageFraction לבין זמן ביצוע שתי השאילתות.

ניתוח התוצאות

הניבה את זמני הריצה הנמוכים ביותר בממוצע (memory.fraction=0.7 –26.69 מהתוצאות עולה כי קבוצת שהניבה את הזמנים הגבוהים ביותר מכל תשע ההרצות (memory.fraction=0.6 28.41 שניות), לעומת 31.77–31.29 שניות). קבוצת 0.8 הייתה ביניים (29.31–29.84 שניות)

עם זמן ריצה של storageFraction=0.6, 26.69 ו-memory.fraction=0.7 הצירוף המהיר ביותר שנמדד היה שניות.

בתוך קבוצת 0.7, שלושת הצירופים היו קרובים יחסית זה לזה (26.69–28.41 שניות), ואילו בקבוצת 0.6 כל שלושת הצירופים היו איטיים יותר באופן עקבי מהצירופים המקבילים בקבוצות 0.7 ו-0.8.

נימוק לתוצאות

גבוהה יותר תשפר תמיד את הביצועים, הממצאים מראים תמונה memory.fraction בניגוד לציפייה שהקצאת הניבה דווקא את זמני הריצה האיטיים (ההקצאה הנמוכה ביותר שנבדקה) memory.fraction=0.6 :מעורבת הניבה את הזמנים המהירים ביותר memory.fraction=0.7 ביותר, בעוד

יתרה מזאת, הקצאה גבוהה יותר (0.8) לא המשיכה לשפר את הביצועים לעומת 0.7, מה שמרמז שקיימת נקודת איזון סביב 0.7 שמעבר לה אין תועלת נוספת.

שווה ל־0.6 סיפק איזון מתאים בין הזיכרון storageFractionשווה ל־0.7 ו־ memory.fraction הצירוף של לבין החלוקה הפנימית בין אחסון לביצוע חישובים. במקרה זה התקבל זמן הריצה הנמוך Sparkהמוקצה ל־ ביותר.

שלב 8 – סיכום:

לצורך סיכום תהליך התחקור רוכזו זמני הביצוע המועדפים שהתקבלו בשלבים 4–7. זמן הבסיס נקבע בשלב 4 והיה 20.06 שניות.

שלב	שם הבדיקה	התצורה המועדפת	זמן ריצה	יחס לשלב הקודם	קיצור לעומת השלב הקודם	יחס לזמן הבסיס
4	אחסון בדיסק	table.parquet (ללא partition)	20.06 שניות	100.00%	0.00%	100.00%
5	אחסון ביניים	persist(MEMORY_ONLY)	17.70 שניות	88.24%	11.76%	88.24%
6	זיכרון ה-JVM	spark.driver.memory = 1g	15.07 שניות	85.14%	14.86%	75.12%
7	רכיבי זיכרון	spark.memory.fraction=0.7, storageFraction=0.6	26.69 שניות	177.11%	-77.11%	133.05%

החישובים שבוצעו:

שלב 5 ביחס לשלב 4

100 × 17.70 / 20.06 = 88.24%

הקיצור בזמן היה:

100 - 88.24 = 11.76%

שלב 6 ביחס לשלב 5

100 × 15.07 / 17.70 = 85.14%

הקיצור בזמן היה:

100 - 85.14 = 14.86%

שלב 7 ביחס לשלב 6

100 × 26.69 / 15.07 = 177.11%

לא היה קיצור בזמן; להפך, נרשמה עלייה משמעותית:

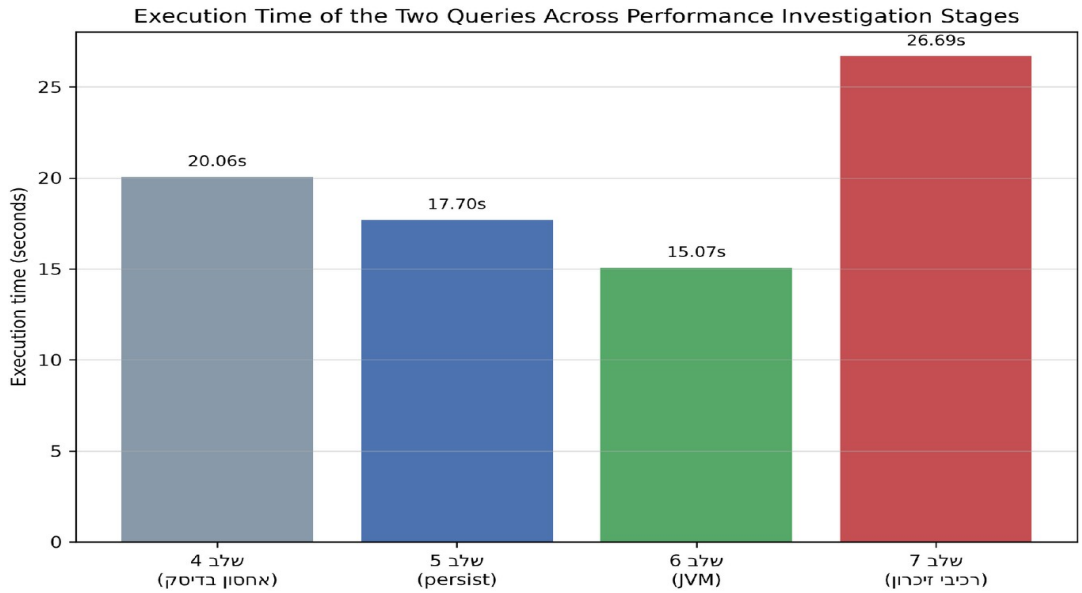
100 - 177.11 = -77.11%

שלב 7 ביחס לזמן הבסיס

100 × 26.69 / 20.06 = 133.05%

כלומר זמן הביצוע הסופי הוא 133.05% מזמן הבסיס, בפועל איטי יותר מזמן הבסיס. במונחי שינוי בזמן, מדובר בעלייה של:

100 - 133.05 = -33.05%



תרשים 3: זמני ביצוע שתי השאילתות לפי שלבי תחקור הביצועים.

מסקנות

בשלב 5 נבדקו שתי אפשרויות של table.parquet. זמן הבסיס שנמדד בשלב 4 היה 20.06 שניות בעת קריאת persist (שיפור של 11.76%) היה מעט מהיר יותר מההרצה ללא persist: MEMORY_ONLY ואילו, היה איטי משמעותית מכל האפשרויות האחרות. בהתאם לתוצאה הטובה ביותר, MEMORY_AND_DISK עודכן להשתמש ב queries.py הסקריפט persist(MEMORY_ONLY)-.

הפחיתה את זמן הביצוע GBהקצאה של 1 .JVM-שיפור נוסף התקבל בשלב 6, שבו נבדקה הקצאת הזיכרון ל 17.70- מ-15.07 שניות. מדובר בקיצור של 14.86% בזמן הביצוע ביחס לשלב הקודם.

בערך 0.7 spark.memory.fraction בשלב 7 בוצע כוונון נוסף של רכיבי הזיכרון. השילוב של בערך 0.6 הניב זמן ריצה של 26.69 שניות, עלייה של 77.11% ביחס לשלב 6 (15.07 storageFraction-ו שניות), ולמעשה זמן ריצה גבוה יותר גם מזמן הבסיס שנמדד בשלב 4.

בהשוואה לזמן הבסיס, התצורה הסופית של שלב 7 הביאה למעשה לזמן ריצה גבוה יותר, מ-20.06 שניות ל- 26.69 שניות, עלייה של 33.05% מזמן הבסיס. מבין כל התצורות שנבדקו, זו עם הזמן הטוב ביותר בפועל הייתה הקצאת driver.memory=1g בשלב 6 (15.07 שניות, 75.12% מזמן הבסיס).

שלב שמירת הביניים (שלב 5) תרם שיפור .(שלב 6) JVM-השלב שהועיל ביותר היה שלב הקצאת הזיכרון ל קל, ואילו כוונון רכיבי הזיכרון בשלב 7 דווקא פגע בביצועים לעומת שלב 6, ממצא שמרמז שההקצאה של לבדה הייתה כבר קרובה לתצורה האופטימלית עבור טבלה קטנה זו, וכוונון נוסף של driver.memory=1g לא הוסיף ערך ואף הזיק memory.fraction/storageFraction.

בחינה ביקורתית של סדר הבדיקות

היה זה שתרם את השיפור העקבי ביותר, בעוד שלב 7 (רכיבי JVM-הקצאת זיכרון ל) בהרצה זו, שלב 6 זיכרון) פגע בביצועים לעומת שלב 6. יש לשים לב שכל שלב נבדק על ידי סקריפט נפרד ובבידוד מהשלבים האחרים, כך שהתוצאות עשויות שלא לשקף את ההשפעה המשולבת של כל הפרמטרים יחד.

עם זאת, יש היגיון בשמירה על סדר התחקור המקורי, משום שכל שלב התבסס על התצורה המועדפת Driverשנבחרה בשלב שקדם לו. לדוגמה, בדיקות רכיבי הזיכרון בשלב 7 בוצעו לאחר שנבחרה הקצאת ה־ המועדפת בשלב 6.

בבדיקות עתידיות ניתן היה להקדיש יותר בדיקות לערכי זיכרון הקרובים לתוצאה הטובה ביותר. לדוגמה, סביב 1 GBאו 1.25 MBהיה הערך המועדף, ניתן היה לבדוק גם ערכים כגון 750 GBלאחר שנמצא כי 1 GB.

באופן דומה, מומלץ לחזור ולבדוק את טווחי spark.memory.fraction ו-storageFraction תוך הפעלת קבוע, כדי לוודא שהתצורה האופטימלית (0.8/0.6) אכן נשארת persist(MEMORY_AND_DISK) המועדפת גם בשילוב עם אחסון הביניים, ולא רק כאשר היא נבדקת בבידוד.

הערה מתודולוגית: הריצה הקודמת של הדוח בוצעה על תיקייה מסונכרנת ל-OneDrive והציגה שונות/רעש משמעותית בין ריצות שונות (נמדד בין 3.74 ל-32.23 שניות למשל) ניכר בין הרצות המספרים בגרסה זו. נמדד מחדש מתיקייה מקומית שאינה מסונכרנת לענן, ומציגים שונות נמוכה משמעותית יותר. עדיין מומלץ להריץ מחדש על הקורס הרשמי לפני ההגשה הסופית VM/Colab-לאמת את המספרים הסופיים בהרצה נוספת.